

HW1 Writeup: Linear and Neural Sentiment Classification

Zhizhe Zhang

1 Implementation Summary

I implemented all required components in `models.py`:

- `UnigramFeatureExtractor`, `BigramFeatureExtractor`, `BetterFeatureExtractor`
- `LogisticRegressionClassifier` and `train_logistic_regression`
- `NeuralSentimentClassifier`, `DANNetwork`, and `train_deep_averaging_network`

For logistic regression, I used sparse bag-of-words features with SGD, random shuffling each epoch, a decayed learning-rate schedule

$$\eta_t = \frac{\eta_0}{1 + 0.05 \cdot \text{epoch}},$$

and a small L2 shrinkage term.

For DAN, each sentence is converted to word indices, embeddings are averaged, and a feed-forward network (`Linear` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout` $\rightarrow$ `Linear`) predicts two classes. Training uses Adam + NLLLoss, and I keep the best dev checkpoint.

2 Required Results

Command	Dev Accuracy	Dev F1	Time
<code>python sentiment_classifier.py --model LR --feats UNIGRAM</code>	0.7718	0.7858	1.47s
<code>python sentiment_classifier.py --model DAN</code>	0.7970	0.8066	16.60s

Table 1: Main required results. Both models exceed the 77% dev accuracy threshold.

3 Exploration 1: LR Learning-Rate Schedules

I compared two initial step sizes (`--lr 0.05` and `--lr 0.2`) for UNIGRAM LR with 25 epochs and tracked:

- average training log-likelihood per epoch
- dev accuracy per epoch

Final dev accuracies:

- `--lr 0.05`: dev acc = 0.7695
- `--lr 0.2`: dev acc = 0.7729

Figure 1 shows that larger learning rate improves the training objective faster, but dev accuracy remains very close between the two settings.

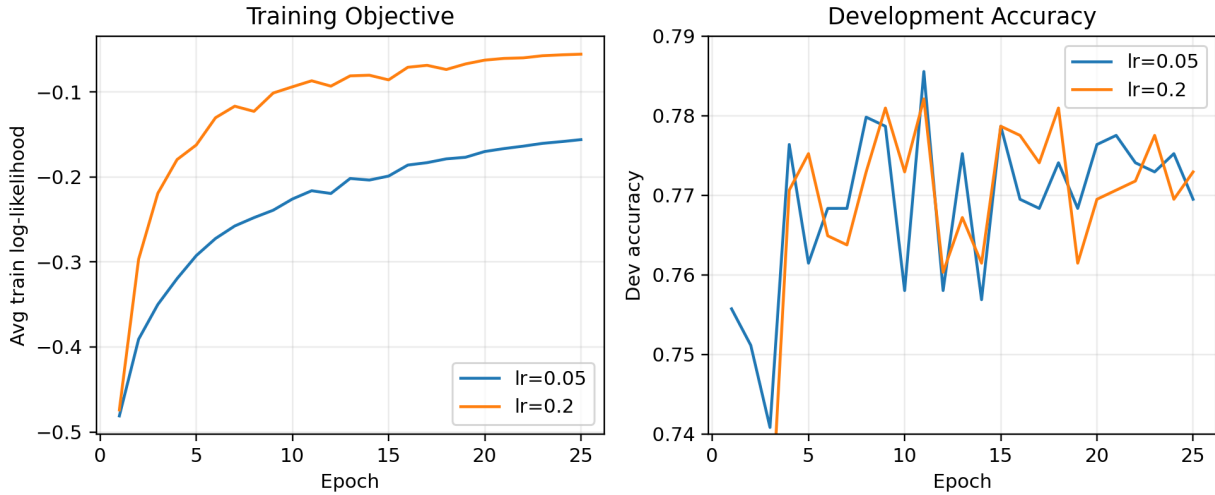


Figure 1: Training objective and dev accuracy versus epoch for two LR schedules.

4 Exploration 2: Feature Modifications

With `--lr 0.1 --num_epochs 25`:

- `--feats BIGRAM`: dev acc = 0.7351
- `--feats BETTER`: dev acc = 0.8085

My **BETTER** extractor is not a unigram+bigram concatenation. It uses:

- stopword filtering and short-token filtering
- clipped unigram counts (1/2 instead of raw counts)
- negation scope marking (next 3 content words get `_NEG`)
- a sentence-level intensifier indicator feature

Observation: BIGRAM overfits strongly, while these modified features improve dev performance.

5 Additional DAN Check

I also compared DAN variants:

- 50d embeddings: dev acc = 0.7431

- 300d embeddings + hidden size 200: dev acc = 0.7936

This suggests that embedding dimensionality (300d vs 50d) has a strong effect, while increasing hidden size alone does not reliably beat the baseline.